ANKARA YILDIRIM BEYAZIT UNIVERSITY

GRADUATE SCHOOL OF NATURAL AND APPLIED SCIENCES

COMPUTER ENGINEERING

DataGen: Matrix Dataset Generation

Spring Term Midterm Report

By

Ali Emre Pamuk 21050111021

Faruk Kaplan 21050111026

Mert Altekın 21050111065

Supervised by

Fahreddin Şükrü Torun

Date

27.03.2025

Abstract

This project addresses the challenges of large-scale matrix generation and optimization by developing a framework for efficient processing, visualization, and analysis. In the recent development phase, significant improvements were made to enhance computational efficiency and scalability. Core operations such as matrix loading, property extraction, and optimization were transitioned to sparse matrix structures, substantially reducing memory usage and runtime. The optimization strategy was refined by prioritizing structural features in the loss function, informed by relevant research. Additionally, new interpolation-based matrix scaling techniques, adapted from image and signal processing, were implemented to generate high-fidelity expanded matrices. To support future machine learning applications, a dataset generation module was introduced using physics-based simulations from fluid mechanics, heat transfer, and solid mechanics. The framework retains its interactive capabilities via a Tkinter-based GUI, continuing to offer intuitive visualization and analysis tools.

Contents

Introduction

The efficient generation and processing of matrices are pivotal in numerous scientific and engineering domains, including data analysis, machine learning, computational simulations, and scientific computing. Matrices serve as fundamental building blocks in these fields, enabling the representation of complex systems and the solving of large-scale computational problems. However, as the scale and complexity of these matrices continue to grow, traditional methods of matrix processing face significant limitations. Challenges such as computational inefficiency, difficulty in preserving critical structural properties, and scalability issues have highlighted the need for more advanced and adaptable solutions.

This research project seeks to address these limitations by developing a robust framework for the generation, manipulation, and optimization of matrices. The framework focuses on creating matrices with varying sizes and complexities while ensuring that essential structural properties, such as sparsity, symmetry, and numerical stability, are preserved. By combining theoretical exploration with practical implementation, the project aims to push the boundaries of computational efficiency and accuracy in matrix processing. The ultimate goal is to provide methods that are both versatile and scalable, capable of addressing the needs of modern computational systems.

To achieve these objectives, the research systematically examines the existing body of knowledge, including algorithms, tools, and techniques for matrix processing. Key references in this domain include foundational works such as:

- **R-MAT Graph Generator:** D. Chakrabarti, Y. Zhan, and C. Faloutsos, R-MAT: A Recursive Model for Graph Mining [1].
- **SPARSKIT:** Youcef Saad's toolkit for sparse matrix computations, which provides practical resources for handling sparse systems [4].
- **Finite Element Matrix Generation on a GPU:** A. Dziekonski, P. Sypek, A. Lamecki, and M. Mrozowski's exploration of GPU-based matrix generation [2].
- **J-Orthogonal Matrices:** Nicholas J. Higham's study on properties and generation methods of orthogonal matrices [3].

These works provided critical insights into the state-of-the-art techniques for matrix generation and manipulation, informing the research methodology and identifying areas for innovation.

The project also involves the collection and exploration of diverse datasets, sourced from publicly available repositories and applications in fields such as multiphysics simulations. In addition to traditional sources, new datasets were generated through physics-based simulations involving fluid dynamics, heat transfer, and solid mechanics, enabling more realistic and diverse matrix structures. The research further includes the design and implementation of advanced matrix manipulation and optimization methods, incorporating sparse matrix representations to improve computational performance. These enhancements also support future integration with artificial intelligence (AI)-based approaches, particularly in generating large-scale datasets for machine learning applications.

Methodology

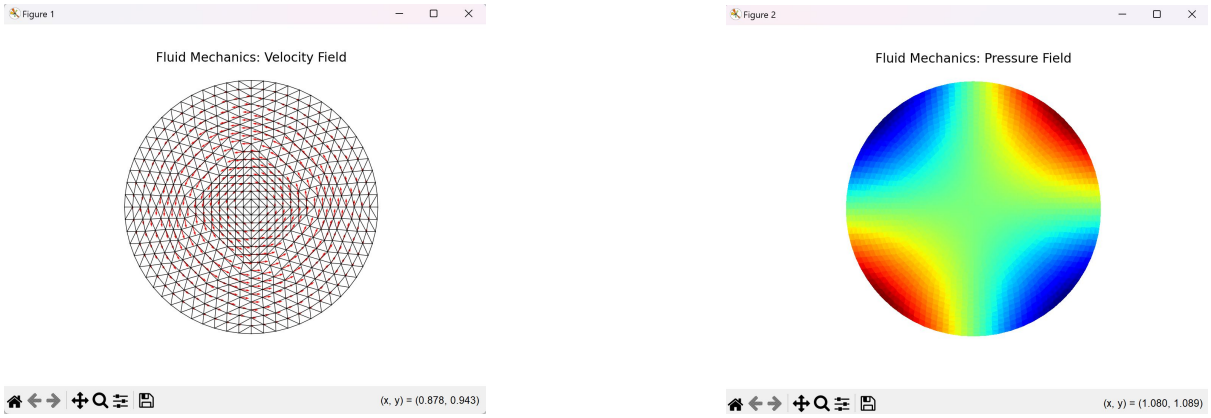
This project utilized a systematic methodology for matrix generation, evaluation, and optimization, designed to address the computational challenges associated with large-scale matrices. The methodology was divided into three main stages: data collection, matrix generation, and optimization. Below, we describe each phase in detail.

Research and Exploration

We began by collecting matrices from public repositories (e.g., SuiteSparse Matrix Collection). To expand the dataset, we generated additional matrices using physics-based simulations in fluid mechanics, heat transfer, and solid mechanics. This was complemented by research into interpolation methods, optimization strategies, and generative models such as GANs, which informed the design of scalable and efficient generation techniques. The resulting matrices encompassed diverse structural and numerical characteristics, forming the foundation for the subsequent phases of the project.

Data Collection

Matrices were sourced from public repositories such as the SuiteSparse Matrix Collection. In addition, custom datasets were generated using finite element simulations of physical systems, including fluid flow 7 and heat conduction 8. This hybrid approach ensured diversity in structure, sparsity, and numerical characteristics, providing a comprehensive basis for generation, analysis, and optimization tasks.



(a) Velocity Field (b) Pressure Field

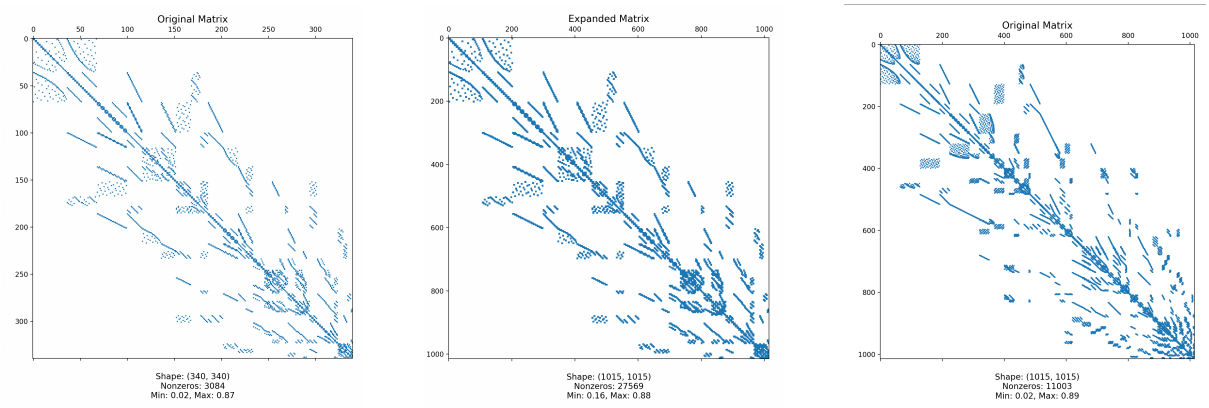
Figure 1: Collected Example Matrices

Matrix Generation

To generate new matrices, we implemented scaling and expansion algorithms. Given an initial matrix $A \in R^{m \times n}$, we used interpolation-based techniques inspired by image processing to create matrices of desired dimensions while preserving structural properties 4. These methods introduced controlled noise and density adjustments to simulate real-world scenarios while maintaining sparsity5. The algorithm for matrix expansion can be expressed as:

$$X_j = \text{expand\_matrix}(A), \quad \forall j \in \{1, \ldots, k\},$$

Where X_j represents j-th generated matrix, and 'expand_matrix' applies proportional scaling and random perturbations to the non-zero entries of A 6

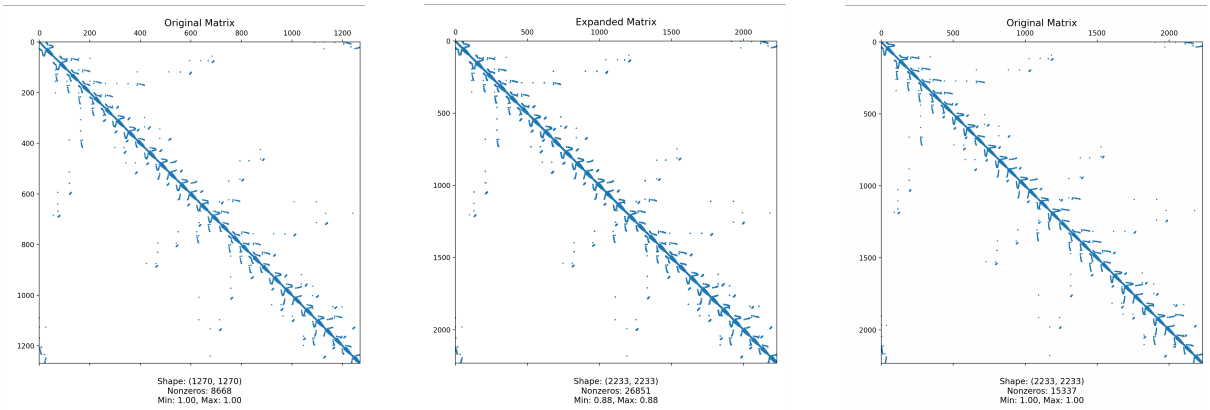


(a) Original Matrix (cage7) (b) Generated (cage8) (c) Original Matrix (cage8)

Figure 2: Generated - Original Comparison (Bi-Linear Interpolation)

To implement scaling and expansion operations on matrices, we leveraged several techniques originally developed for image processing, including nearest-neighbor interpolation 4, bilinear interpolation, and random interpolation 5. Initially, these methods were applied on matrix visualizations, treating the matrix as a grayscale image where each element corresponds to a pixel intensity. This allowed us to experiment with resizing techniques in a visual and intuitive manner, evaluating their effect on matrix structure and distribution.

Following successful trials on visual representations, we re-implemented these interpolation techniques to operate directly on sparse matrices stored in `.mtx` (Matrix Market) format. This transition required careful handling of sparsity to ensure that the expanded matrices preserved both the structural sparsity pattern and essential data characteristics. In particular, we extended standard interpolation algorithms to selectively interpolate only non-zero elements, introducing controlled noise or density modulation when necessary. The result is a set of scalable matrix manipulation tools capable of simulating realistic, high-dimensional sparse datasets for downstream tasks such as benchmarking or training machine learning models.



(a) (lshp1270) (b) Generated (lshp2233) (c) (lshp2233)

Figure 3: Generated - Original Comparison (Nearest-Neighbor Interpolation)

Optimization

The optimization phase aimed to minimize the difference between the original matrix A and the generated matrices X_j in terms of specific numerical and structural properties. To improve performance, all operations were refactored to use sparse matrix representations, reducing memory and computational overhead. Additionally, the loss function was adjusted to place greater emphasis on structural characteristics such as sparsity patterns and symmetry, guided by findings from recent research on matrix structure importance [**?**]. This shift ensured that optimization focused not only on numerical alignment but also on preserving structural integrity across generated matrices.

$$P(A) = \{p_1(A), p_2(A), \ldots, p_r(A)\}$$

where each property $p_i(A)$ represents a numerical measure, such as:

- **Non-zero statistics**: $\min(\text{nnz}(A_i;:)), \max(\text{nnz}(A_i;:)), \text{mean}, \text{std}$,
- **Symmetry checks**: Pattern symmetry and numerical symmetry,
- **Matrix norms**: $\|A\|_1, \|A\|_\infty, \|A\|_F$,
- **Condition number**: $\text{cond}(A, 1)$.

To evaluate the quality of generated matrices X_j, we applied *min-max scaling* to normalize the properties:

$$p_i^{\min} = \min_j p_i(M_j), \quad p_i^{\max} = \max_j p_i(M_j)$$

resulting in a scaling dictionary $\mathcal{S}$ with bounds for each property p_i.

A property-based loss function was used to quantify the difference between A and X_j:

$$\text{Loss}(A, X_j) = \sum_{i=1}^{r} w_i \cdot \Delta_i(A, X_j)$$

where $\Delta_i(A, X_j)$ measures the scaled absolute difference of property p_i between A and X_j, defined as:

$$\Delta_i(A, X_j) = \begin{cases} \frac{|p_i(X_j) - p_i(A)|}{p_i^{\max} - p_i^{\min} + \epsilon}, & \text{if } p_i^{\max} \neq p_i^{\min}, \\ 0, & \text{otherwise.} \end{cases}$$

Here, w_i is a weight assigned to each property, and $\epsilon > 0$ prevents division by zero.

Finally, we minimized the total loss across all generated matrices using *local search optimization*:

$$\text{TotalLoss}(A, X_1, \ldots, X_k) = \sum_{j=1}^{k} \text{Loss}(A, X_j)$$

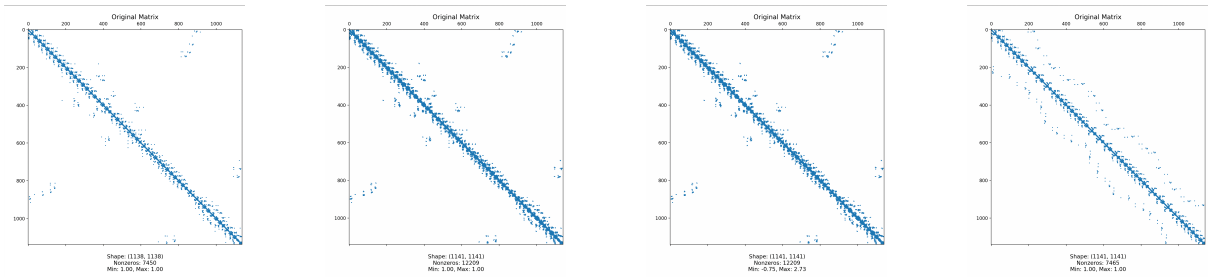
The optimization process iteratively perturbed the matrices X_j to improve their alignment with A. Random perturbations were applied to the non-zero entries, and changes were accepted if they reduced the total loss. This process is summarized as:

1. **Initialization**: Start with matrices $\{X_1, \ldots, X_k\}$ from the generation phase.
2. **Iteration**:

- Select X_j randomly.
- Apply a perturbation: $\tilde{X}_j = \text{Perturb}(X_j)$ 1 2.
- Compute the new total loss: L_{new}.
- Accept $\tilde{X}_j$ if $L_{\text{new}} < L_{\text{old}}$, otherwise revert.

3. **Output**: Optimized matrices $\{\tilde{X}_1, \ldots, \tilde{X}_k\}$ 3.

This iterative process ensured gradual exploration of the solution space while maintaining computational efficiency.



(a) jagmesh7 (b) Generated (c) Optimized (d) jagmesh8

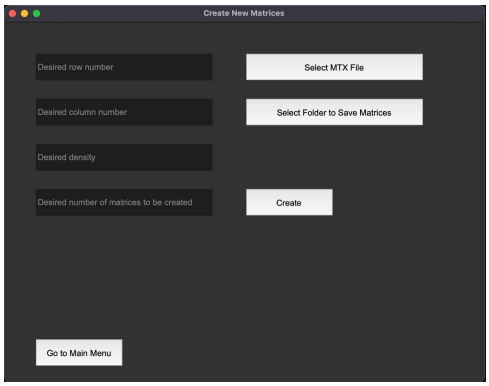
Figure 4: Producing Jagmesh8 from Jagmesh7

Graphical User Interface (GUI) Development

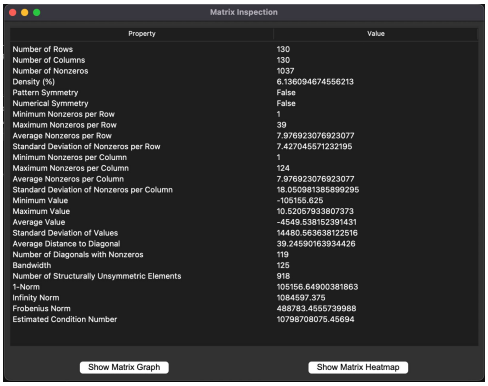
To streamline and simplify the workflow described in the previous sections, we developed a user-friendly Graphical User Interface (GUI). The GUI serves as a comprehensive tool for executing matrix generation, evaluation, and optimization tasks seamlessly. Key features of the GUI include:

- **Matrix Import and Visualization:** Users can import matrices and visualize their structure, including sparsity patterns and numerical properties.
- **Interactive Matrix Generation:** The interface allows users to specify basic parameters for matrix generation.
- **Property Analysis and Comparison:** The GUI computes and displays numerical properties of matrices, enabling easy comparison between original and generated matrices.
- **Export and Integration:** Generated and optimized matrices can be exported for use in external applications or simulations, ensuring compatibility with existing workflows.

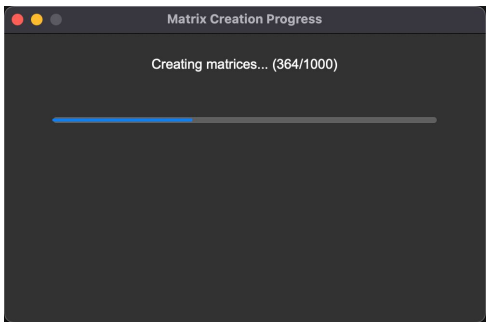
The GUI was built using Tkinter/Python to ensure user-friendly experience. This tool enhances accessibility, enabling both researchers and practitioners to efficiently apply the developed methodology without delving into the underlying codebase.



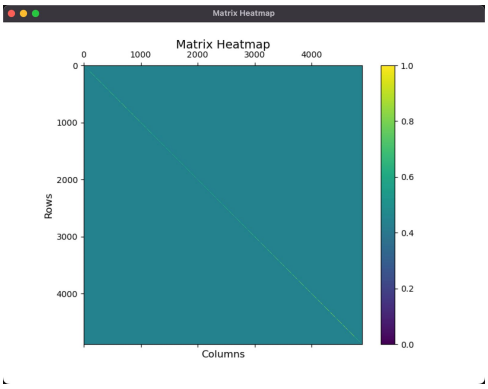
(a) Creation Page of GUI



(b) Features Menu



(c) Progress Bar



(d) Heat Map

Figure 5: Glance to GUI

Challenges and Limitations

Throughout the course of this project, we encountered several challenges and limitations, ranging from technical implementation to computational constraints. These issues were addressed systematically, but they highlight areas where further refinement or resources could improve the framework. Below, we categorize and detail the challenges:

Matrix Collection and Preparation

- **Collecting matrices from various sources:** Locating diverse and high-quality matrices with specific properties was time-intensive and required extensive exploration of repositories and applications such as FEATools Multiphysics.
- **Converting data formats to matrices**: Many datasets were stored in non-matrix formats, necessitating preprocessing and conversion to extract usable matrices.

Feature Engineering and Property Calculation

- **Finding matrix features:** Identifying key numerical and structural properties for analysis required significant effort in feature engineering.
- **Mathematically and numerically calculating property values:** Developing robust algorithms to compute properties such as symmetry, sparsity, and norms

was challenging, particularly for large-scale matrices.

- **Encoding categorical features:** Converting categorical properties into numerical values for use in property-based loss calculations was complex and required careful encoding strategies.

Matrix Generation and Optimization

- **Generating expanded matrices:** Scaling matrices, shifting their values, and adjusting their density while preserving key structural properties proved to be computationally demanding.
- **Finding an appropriate loss function:** Designing a property-based loss function that effectively measured the difference between the original and generated matrices required extensive experimentation.
- **Decreasing loss and improving matrix similarity:** Iteratively minimizing the property-based loss to align the generated matrices with the original matrix presented challenges in terms of computational efficiency and convergence.
- **Handling numeric errors:** Issues such as NaN values, infinite values, and memory overflows frequently arose during calculations, requiring additional error-handling mechanisms.

Computational and Resource Constraints

- **Intensive memory and computational resource needs:** The large-scale nature of the matrices required significant computational power and memory, which constrained some aspects of the project.
- **Effective resource usage:** Efficiently managing system resources, such as memory, processing power, and Tkinter objects for the GUI, was a persistent challenge.

Graphical User Interface (GUI) Development

- **Connecting frontend inputs to backend operations:** Integrating user inputs and file uploads with backend processing required careful design and debugging.
- **Adding graphs and heatmaps:** Visualizing matrix properties and patterns in the GUI using heatmaps and graphs demanded both computational resources and responsive design techniques.
- **Effective responsive design:** Ensuring that the GUI was user-friendly, responsive, and functional across different screen sizes and resolutions was a key challenge.

Application-Specific Issues

- **Installation and usage of simulation applications:** Tools such as ANSYS required substantial setup and learning, adding to the complexity of data collection and preprocessing.

- **Limited resources for similar implementations:** The lack of existing implementations and resources specific to this type of matrix generation and optimization project required innovative problem-solving and original methods.

Results

The results of this research project demonstrate the successful implementation and execution of the proposed methodology, achieving the primary objectives. Key outcomes are summarized as follows:

Graphical User Interface (GUI)

- The Tkinter-based GUI remained fully functional and was updated to support the new sparse matrix operations.
- Input/output handling was adapted for efficient processing of large sparse datasets, and visualization components were refined to display structural patterns more effectively.

Matrix Collection and Generation

- Matrices were collected from public repositories and also generated using physics-based finite element simulations, resulting in a richer and more diverse dataset.
- New scaling and expansion algorithms, including interpolation-based methods, produced structurally consistent matrices across varying dimensions.

Feature Extraction and Visualization

- Key features such as symmetry, sparsity, and numerical distributions were accurately extracted from both generated and original matrices.
- Visualization tools were enhanced to accommodate sparse matrix patterns using heatmaps and structural overlays within the GUI.

Optimization and Loss Reduction

- The updated optimization process using sparse matrices led to significant improvements in performance and stability.
- Structural weights were applied to the loss function, resulting in better alignment of generated matrices in terms of sparsity and pattern similarity.
- The new perturbation-based approach reduced overall loss while maintaining key matrix properties.

Resource Efficiency

- Memory usage and runtime were substantially reduced through the adoption of sparse matrix formats.
- Optimization and generation routines were restructured to minimize overhead, enabling the processing of larger and more complex matrices without performance degradation.

Future Work

While this project successfully achieved its primary objectives, there are several areas for further development and improvement. The planned future work aims to expand the functionality of the framework, enhance its performance, and explore advanced applications. The key directions are as follows:

Deep Learning Integration

- Transitioning to the machine learning phase, we plan to train models using the matrices generated during this project. This will enable predictive and automated insights into matrix generation, optimization, and property analysis.

Graphical User Interface (GUI) Enhancements

- The GUI will be improved to incorporate advanced features such as responsive design for various screen resolutions and selective resolution settings, ensuring a better user experience.
- Enhanced visualization tools, including more detailed and interactive graphs and heatmaps, will be integrated to provide deeper insights into matrix properties and patterns.

Code Quality and Modularity

- The codebase will be refined to enhance modularity, memory efficiency, and execution speed. This includes restructuring key components and optimizing algorithms for scalability.

Optimization Improvements

- More deterministic optimization techniques will be implemented to reduce the reliance on random and stochastic processes. These techniques will aim for greater consistency and reliability in achieving optimal solutions.

References

[1] Deepayan Chakrabarti, Yiping Zhan, and Christos Faloutsos. R-mat: A recursive model for graph mining. *SIAM International Conference on Data Mining*, 2004. Accessed from foundational works on graph mining.

[2] Andrzej Dziekonski, Piotr Sypek, Andrzej Lamecki, and Miroslaw Mrozowski. Finite element matrix generation on a gpu. *Computer Methods in Applied Mechanics and Engineering*, 254:224–232, 2013. Exploration of GPU-based matrix generation.

[3] Nicholas J. Higham. J-orthogonal matrices: Properties and generation methods. *SIAM Journal on Matrix Analysis and Applications*, 23(4):1124–1135, 2002. Study on orthogonal matrices.

[4] Youcef Saad. *SPARSKIT: A Basic Tool Kit for Sparse Matrix Computations*. University of Minnesota, 1994. Toolkit for handling sparse matrix computations.

Appendix

Algorithm 1 Perturb Nonzero Values of a Matrix

Input: Sparse matrix $matrix$, perturbation range ϵ
Output: Perturbed matrix *perturbed*
Create a copy of $matrix$ as *perturbed*
Generate random values *perturbation* in range $[1-\epsilon, 1+\epsilon]$
Multiply nonzero data in *perturbed* by *perturbation*
Return: *perturbed*

Algorithm 2 Perturb Nonzero Positions in Sparse Matrix

Input: Sparse matrix $matrix$, max offset max_offset, fraction f
Output: Perturbed matrix *perturbed*
Convert $matrix$ to COO: $coo \leftarrow matrix.tocoo()$
Select $f\times$ (nonzeros) $\rightarrow indices$
Copy: $rows \leftarrow coo.row$, $cols \leftarrow coo.col$
Generate random offsets in $[-max\_offset, max\_offset]$
Apply and clip offsets: $rows[indices]$, $cols[indices]$
Build new COO with updated $(rows, cols)$ and original data
Return: CSR of new COO $\rightarrow$ *perturbed*

Algorithm 3 Optimize Sparse Matrix by Perturbing Values and Positions

Input: Sparse matrix $matrix$, value perturbation range ϵ, max position offset max_offset, perturbation ratio $perturb\_fraction$
Output: Optimized (perturbed) matrix
$matrix \leftarrow perturb\_values(matrix, \epsilon)$ ▷ Apply value perturbation
$matrix \leftarrow perturb\_positions(matrix, max\_offset, perturb\_fraction)$ ▷ Apply position perturbation
Return: $matrix$

Algorithm 4 Nearest Neighbor Interpolation for Sparse Matrix Expansion

Input: Sparse matrix A, target size (R', C')
Output: Interpolated matrix A'
if A not CSR **then**
    Convert: $A \leftarrow csr\_matrix(A)$
end if
$(R, C) \leftarrow A.shape$
$(r\_scale, c\_scale) \leftarrow (R'/R, C'/C)$
Extract nonzeros: $(i, j) \leftarrow A.nonzero()$, $v \leftarrow A.data$
Scale indices: $(i', j') \leftarrow (\lfloor i \cdot r\_scale \rfloor, \lfloor j \cdot c\_scale \rfloor)$
Clip (i', j') to bounds $(R'-1, C'-1)$
Repeat v to match (i', j') if needed
$A' \leftarrow$ CSR matrix from (i', j', v)
Return: A'

Algorithm 5 Random Sparse Expansion of a Matrix

Input: Sparse matrix A, target size (R', C'), flag *keep_density*
Output: Expanded sparse matrix A'
if A not CSR **then**
    Convert to CSR
end if
$(R, C) \leftarrow A.shape$, $(r_s, c_s) \leftarrow (R'/R, C'/C)$
$(i, j) \leftarrow A.nonzero()$, $v \leftarrow A.data$
Compute number of duplicates based on *keep_density*
Generate random offsets: $(\Delta i, \Delta j)$
$i' \leftarrow$ repeated $i + \Delta i$, clipped to $[0, R' - 1]$
$j' \leftarrow$ repeated $j + \Delta j$, clipped to $[0, C' - 1]$
Repeat v to match (i', j')
$A' \leftarrow csr\_matrix((v, (i', j')), shape = (R', C'))$
Return: A'

Algorithm 6 Blockwise Expansion of a Sparse Matrix

Input: Sparse matrix *original_matrix*, expansion factors row_factor, col_factor
Output: Blockwise expanded sparse matrix *expanded_matrix*
if *original_matrix* is not in CSR format **then**
    Convert to CSR: $original\_matrix \leftarrow csr\_matrix(original\_matrix)$
end if
Get original shape: $(old\_rows, old\_cols) \leftarrow original\_matrix.shape$
Compute new shape: $(new\_rows, new\_cols) \leftarrow (old\_rows \times row\_factor, old\_cols \times col\_factor)$
Initialize empty COO matrix: $expanded\_matrix \leftarrow coo\_matrix((new\_rows, new\_cols))$
for $i \leftarrow 0$ to $row\_factor - 1$ **do**
    for $j \leftarrow 0$ to $col\_factor - 1$ **do**
        $row\_offset \leftarrow i \times old\_rows$
        $col\_offset \leftarrow j \times old\_cols$
        $(block\_rows, block\_cols) \leftarrow original\_matrix.nonzero()$
        $values \leftarrow original\_matrix.data$
        $new\_block\_rows \leftarrow block\_rows + row\_offset$
        $new\_block\_cols \leftarrow block\_cols + col\_offset$
        Add new block to matrix:
        $expanded\_matrix \mathrel{+}= coo\_matrix((values, (new\_block\_rows, new\_block\_cols)), shape = (new\_rows, new\_cols))$
    end for
end for
Convert to CSR format: $expanded\_matrix \leftarrow expanded\_matrix.tocsr()$
Return: *expanded_matrix*

Algorithm 7 FEM Solution of Stokes Flow

Output: *mesh*, *basis*, *velocity*, *pressure*, ψ
Create circular mesh: *mesh* $\leftarrow$ MeshTri.init_circle(4)
Define elements: P2 for velocity, P1 for pressure
Build basis functions: *basis*
Assemble matrices: A (Laplacian), B (divergence), C (pressure mass)
Build saddle-point system:

$$K = \begin{bmatrix} A & -B^T \\ -B & 1e^{-6}C \end{bmatrix}$$

Assemble RHS f (body force in y-direction)
Apply BCs and solve: $uvp \leftarrow \text{solve}(K, f)$
Split: *velocity*, *pressure* $\leftarrow \text{split}(uvp)$
Compute stream-function ψ:
Build Laplacian A_ψ, compute vorticity from *velocity*
Solve: $A_\psi \psi =$ vorticity
Diagnostics: $\max |\vec{u}|$, pressure range, ψ range
Visualize: mesh, $\vec{u}$ (quiver), pressure (contour), ψ (contour)
Return: *mesh*, *basis*, *velocity*, *pressure*, ψ

Algorithm 8 Crank–Nicolson Method for Transient Heat Equation

Output: *mesh*, *basis*, final temperature T
Create unit square mesh
Define P1 finite element basis
Set diffusivity $\kappa = 1.0$
Assemble: K (stiffness), M (mass)
Apply Dirichlet BCs: $T = 0$ on boundary
Set parameters: $\Delta t = 0.01$, $t_{\text{end}} = 0.5$, $\theta = 0.5$
Init: $T(x, y, 0) = \cos(\pi x)\cos(\pi y)$
Time matrices:

$$A = M + \theta \Delta t K, \quad B = M - (1 - \theta)\Delta t K$$

LU factorize A
for each time step **do**
    $RHS \leftarrow B \cdot T$
    Solve $A \cdot T_{\text{new}} = RHS$
    Update $T \leftarrow T_{\text{new}}$, apply BCs
end for
Visualize final T (contour plot)
Return: *mesh*, *basis*, T